

Intelligent Support Ticket Resolution Assistant

A telecom support agent pastes a customer complaint and gets labels, the most similar past cases, and a drafted step-by-step fix in which every step names the ticket or article it came from. It searches by meaning, not only by keyword, and says “escalate” when it finds nothing good enough.

Unnathi Vinod · github.com/unnathivinod/ticket-resolution-assistant-

What the agent gets in one screen

- 1. **What it is:** category, product, severity and sentiment.
- 2. **What was done before:** similar past tickets and knowledge-base articles, found by meaning and by keyword.
- 3. **What to do now:** a drafted fix. Every step cites its source, and each citation is checked.

The numbers, measured by scripts in the repository

55% vs 40%

the right ticket ranked first: search by meaning against keyword search

100%

of steps backed by their source, in answers to known problems

77%

of off-topic questions stopped before any language model is asked

15 of 20

known problems answered correctly by the hosted model on Groq (11 by the local one)

71%

of simulated outages noticed, with 2% false alarms

363 + 24

fast tests and tests against the running system, run on every push

What the brief asked for, and where it is

DELIVERABLE	WHERE
Architecture diagram	Pages 3 and 4 here; docs/ARCHITECTURE.md , section 3
Full executable code	The repository. Five commands start everything (page 10)
Additional exploration	Reply to the customer, incident detection, sign-in with roles and cases, new-class discovery (pages 5 and 6)
Evals on system health	Six eval scripts, 19 tested alerts, a 28-panel dashboard (pages 7 and 9)
Production scale considerations	Page 9; docs/ARCHITECTURE.md , section 9
Evolving data and ticket classes	New data is searchable in seconds; new classes are proposed and approved (page 6)

What the agent sees

One complaint, pasted as the customer wrote it. Labels, a cited fix and the sources come back on one page.

Resolve a complaint

Paste what the customer wrote, then press Resolve

My broadband drops every evening around 8 and I've already restarted the router twice, I work from home and this is costing me

Resolve →

CATEGORY

Connectivity intermittent

92% of similar tickets agree

PRODUCT

Broadband

100% of similar tickets agree

SEVERITY

High

Business impact, money pressure

SENTIMENT

Neutral

No strong tone found

Suggested resolution

✓ 4 of 4 steps backed by a source

Give feedback

LIKELY CAUSE

The likely cause is peak-hour congestion on the local exchange causing the line to lose sync between 7pm and 10pm.

Already tried, not suggested again

Restarted the router twice

1

Run a remote line test and compare sync speed and error counts between daytime and 7pm to 10pm.

From KB-001 T-000481 T-000505 T-000511

2

Check the exchange utilisation report for the customer's cabinet to confirm peak-hour congestion.

From KB-001 T-000481 T-000505 T-000511

3

Change the router Wi-Fi channel to a less crowded one to rule out local interference.

From KB-001 T-000481 T-000505 T-000511

4

If the cabinet is over capacity, raise a capacity ticket with the network team and give the customer the expected upgrade date.

From KB-001 T-000481 T-000505 T-000511

Review the draft, then record how the case ended.

→ Escalate to second line

✓ Mark as resolved

Reply to the customer

A message you can edit and send, built only from the checked steps.

Draft reply to customer

Sources found

5 - best match first - click a row to read it

KB-001	Article	Evening broadband drops caused by peak-hour congestion	0.80	Used
T-000481	Past ticket	Evening broadband drops caused by peak-hour congestion	0.91	Used
T-000505	Past ticket	Evening broadband drops caused by peak-hour congestion	0.91	Used
T-000511	Past ticket	Evening broadband drops caused by peak-hour congestion	0.89	Used
KB-002	Article	Random broadband disconnects caused by faulty line filter or internal wiring	0.76	Not used

The Resolve page. "Restarted the router twice" is recognised as already tried and is not suggested again.

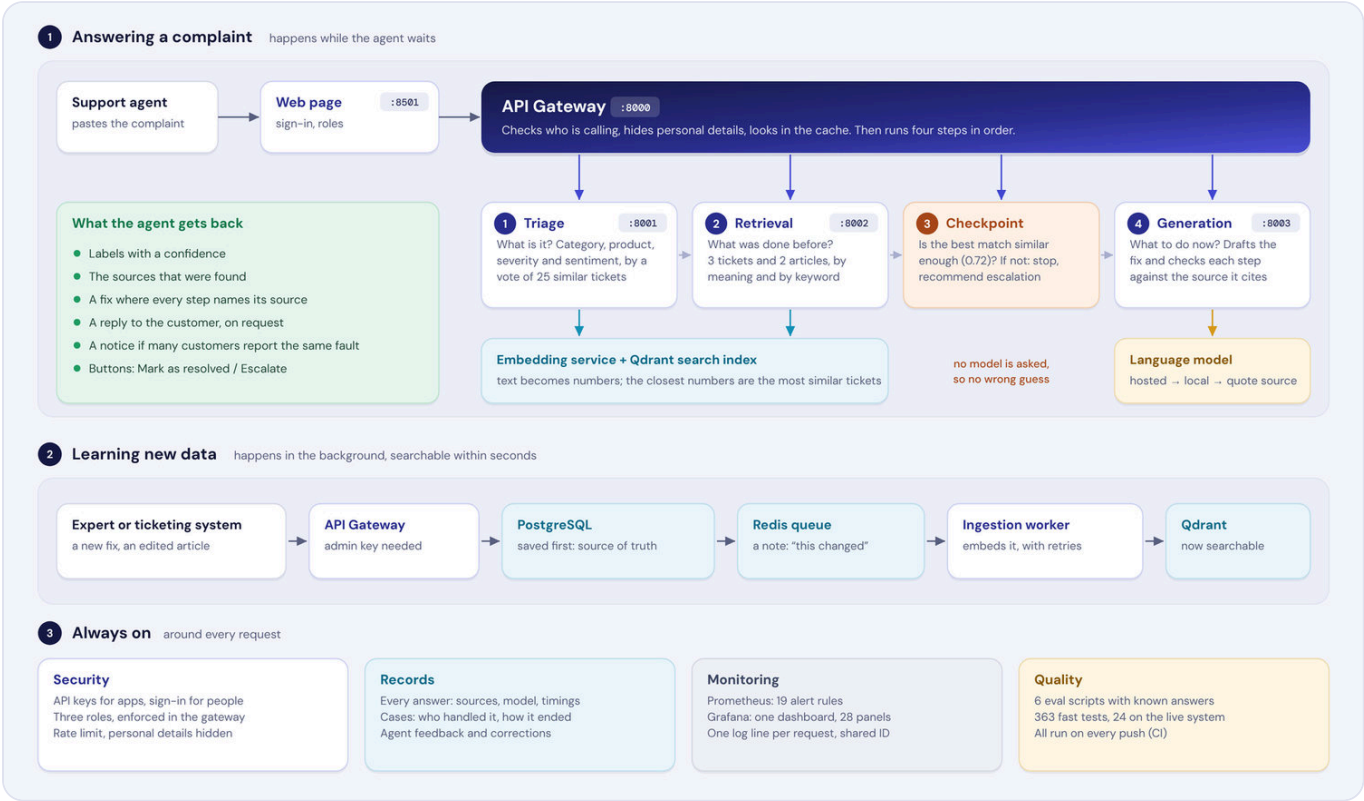
- **Labels** come from a vote among the most similar past tickets, each with a confidence.
- **Every step** shows the tickets and articles it was taken from. Steps that fail the source check are flagged.
- **The agent decides.** Mark as resolved or escalate; the assistant only suggests.

Intelligent Support Ticket Resolution Assistant

2 / 10

How it works

Two paths: answering a complaint while the agent waits, and learning new data in the background.

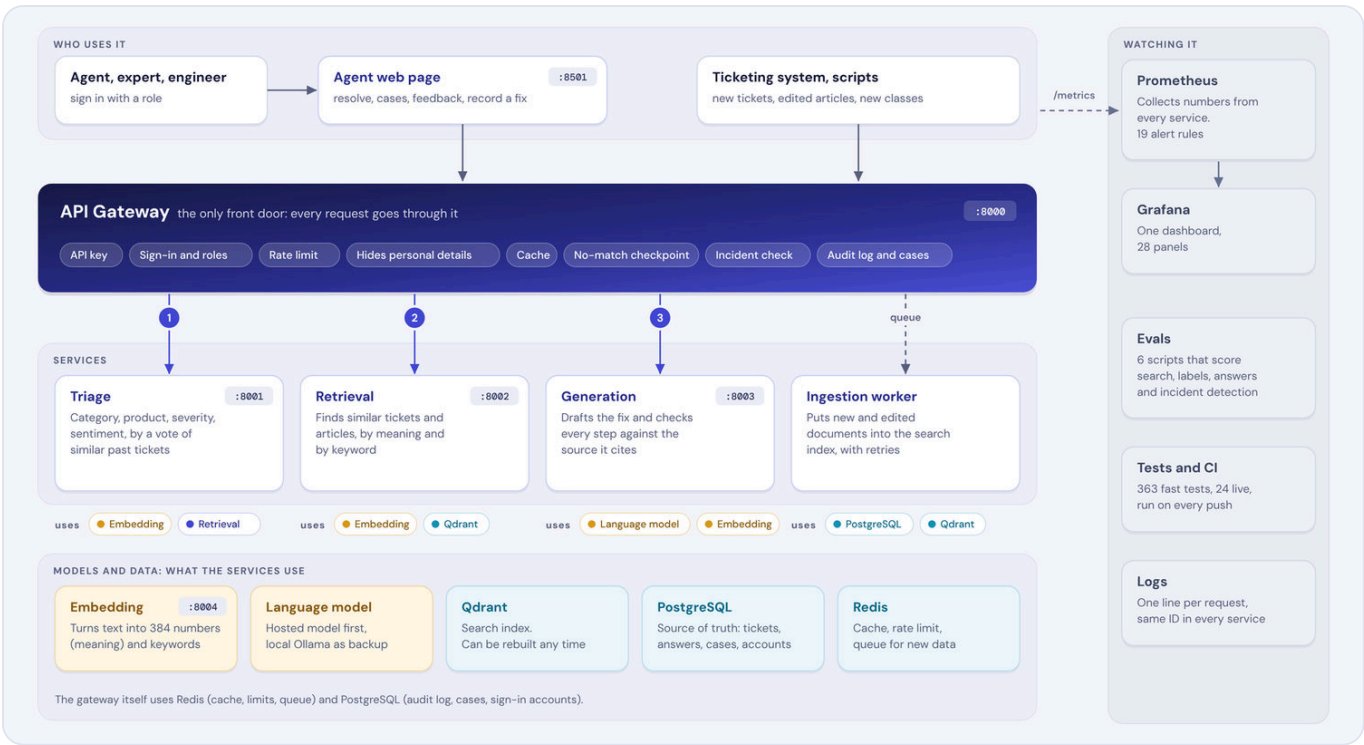


The path of a complaint, the path of new data, and what runs around every request.

STEP	WHAT HAPPENS	TYPICAL TIME
1	The gateway checks the caller and masks emails, phone and account numbers	instant
2	Triage labels the complaint by a vote among the most similar past tickets	under 1 s
3	Retrieval finds the 3 closest tickets and 2 closest articles	under 1 s
4	If nothing is similar enough, it stops here and recommends escalation	
5	The language model drafts a fix. Every step must cite a source, and each citation is checked	31 s local (Ollama), 1.4 s hosted (Groq)

Architecture

Seven small services behind one gateway, with ready-made infrastructure for search, storage, caching and monitoring.



Every request enters through the gateway, which calls the services in the order of the numbers. The dashed line is the queue for new data.

SERVICE	PORT	ITS ONE JOB
Gateway	8000	The only front door: API keys, sign-in and roles, rate limit, masking, cache, checkpoints, audit log, cases
Triage	8001	Category, product, severity, sentiment. Says “unknown” when unsure
Retrieval	8002	Finds the most similar tickets and articles; counts similar recent complaints
Generation	8003	Builds the prompt, calls the model, checks the answer. Backup model, then quotes the source
Embedding	8004	Turns text into numbers that capture meaning, and into keywords
Ingestion worker		Keeps the search index in step with the database, with retries and a safety sweep
Web page	8501	Sign in, resolve, draft the reply, record the decision, list the cases, give feedback

PostgreSQL is the source of truth, Qdrant is the search index (rebuildable at any time), Redis holds the cache, the rate-limit counters and the queue. The language model is a setting: a hosted model first, local Ollama as backup, and no paid API is needed to run the project.

Beyond the brief

Four additions, each built on the parts the brief asked for.

1. A reply to the customer, in one click

Written only from the steps that passed the source check. It apologises if the customer was upset, does not ask them to repeat what they already tried, and contains no ticket numbers.

Reply to the customer

Built from 4 checked steps

Edit

Hello,

We're looking into the evening drop-outs you're experiencing. First, we'll run a remote line test to compare your sync speed and error counts during the day and between 7 pm and 10 pm. Next, we'll check the exchange utilisation report for your cabinet to see if peak-hour congestion is the cause. We'll also change your router's Wi-Fi channel to a less crowded one to rule out local interference. Finally, we may move you to a less loaded port or apply a congestion-relief profile.

We'll keep you updated as we progress. Thank you for your patience.

Kind regards,
[Agent name]

Read it before sending. The copy button is at the top right of the message.

Write it again

The drafted reply, ready to edit and copy.

2. Possible service incident

One customer is a ticket; many customers with the same fault within minutes is an outage. Complaints that mean the same and arrive close together raise a notice on the page and an alert for the person on call.

Resolve a complaint

Paste what the customer wrote, then press Resolve

The internet has been down since this morning in Adyar and my router is showing a red light.

Resolve →

Possible service incident. 7 similar complaints were received in the last 30 minutes.

Hide

25 min ago

Internet is not working in Adyar since early this morning, the router has a red light.

25 min ago

Since this morning the internet is completely down in Adyar and the router light has turned red.

25 min ago

My broadband has been down since this morning in Adyar, the light on the router is red.

CATEGORY

Connectivity outage

85% of similar tickets agree

PRODUCT

Broadband

74% of similar tickets agree

SEVERITY

High

No urgency signals

SENTIMENT

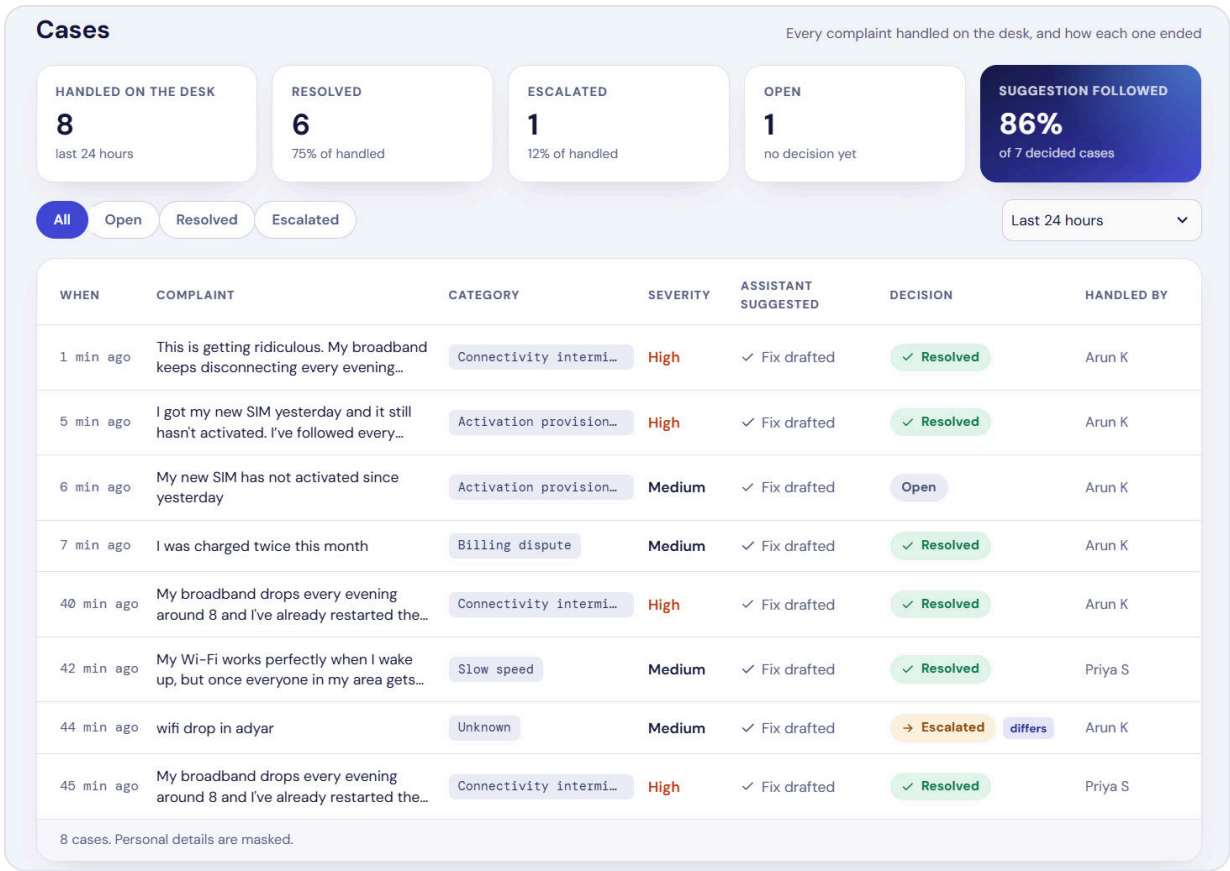
Neutral

No strong tone found

The notice, opened to show the other complaints.

3. Sign-in, roles and cases

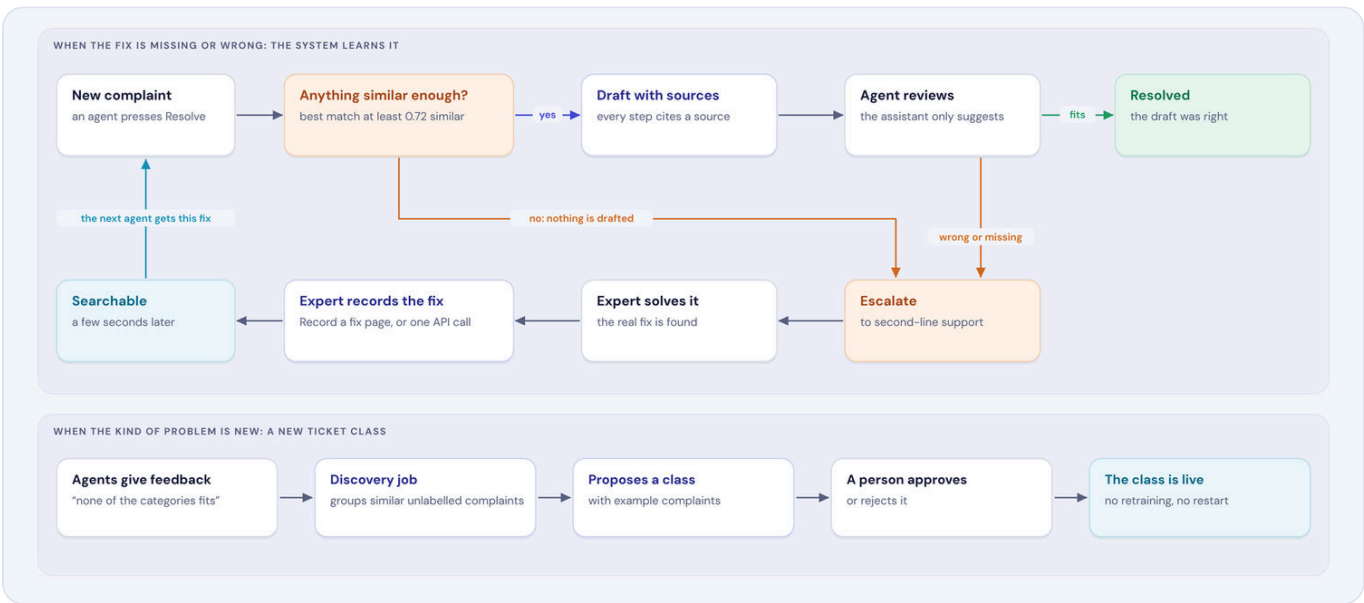
People sign in. An agent sees only the cases they handled; a second-line expert or an engineer sees the whole desk. The rule is enforced in the gateway, not in the page. Every case records what the assistant suggested and what the person decided, so the system measures how often its suggestions are followed.



The Cases page of an expert. One case is marked “differs”: a fix was drafted, and the agent escalated instead.

4. Learning what it did not know

When nothing fits, the complaint is escalated. The expert records the real fix on the page or by one API call, and it is searchable a few seconds later. When agents keep saying “none of the categories fits”, a job groups those complaints and proposes a new ticket class for a person to approve.



The loop for a missing fix, and the path to a new ticket class.

What was measured

Every number comes from a script in `evals/` and can be reproduced. The test complaints use wording that never appears in the indexed tickets, so this is the hard case.

QUESTION	RESULT
Does search by meaning beat keyword search?	The right ticket comes first for 55% of complaints, against 40% for keywords
How good are the labels?	Category 71%, product 78%, sentiment 83%, severity 59% exact and 93% within one level
How fast does new data arrive?	A new ticket is searchable a few seconds after one API call. Nothing is retrained or restarted
Can it learn a new kind of problem?	Search: yes, from a handful of tickets. Category: 18 of 20 for one new class, 1 of 20 for one that overlaps existing classes
Is every step backed by its source?	100% of steps in answers to known problems
Are off-topic questions stopped?	77% by the similarity cut-off, before any model is asked
Is an outage noticed?	Seven customers reporting one fault among twenty other complaints are flagged in 71% of cases; 2% of quiet half hours are flagged by mistake

Two language models on the same 56 complaints

	LOCAL LLAMA3.2:3B ON OLLAMA	HOSTED GPT-OSS-20B ON GROQ
Known problems answered correctly (of 20)	11	15
When the search had found the right source (15)	11 right, 3 mixed, 1 wrong	15 of 15 right
When the search had missed it (5)	5 wrong answers	3 wrong, 2 refused
Problems with no fix in the knowledge base (6)	6 wrong answers	3 wrong, 3 refused
Off-topic questions that reached the model (7)	7 answered	7 refused
Typical time per drafted answer	31 s	1.4 s

With the larger model, every remaining wrong answer on a known problem is a search miss. The model is no longer the weak link; the search is. These are single runs on small groups: a direction, not exact rates.

Design decisions

The main choices, what was considered instead, and why. The full list with the measurements is in `docs/DESIGN_DECISIONS.md`.

DECISION	CHOSEN	INSTEAD OF	WHY
Knowledge	Search first, then let the model write only from what was found	Fine-tuning a model on the tickets	New tickets arrive every day. Search picks them up in seconds; retraining would not
Front door	One gateway calls the services in order	The web page calls each service	One place for security, limits, caching and logging
Labels	A vote among the most similar past tickets	Asking the language model to classify	Fast, explains itself with a confidence, and needs no retraining when a class is added
When nothing fits	A similarity cut-off stops before the model	Always draft an answer	A confident wrong fix costs more than no fix
Storage	PostgreSQL holds the truth, Qdrant is a search index	Keeping everything in the vector database	The index can be rebuilt at any time, and a missed update is repaired from the database
Cache	Exact match on the masked complaint, one hour	A cache for similar complaints	The same text always got the same sources. A similar-text cache can serve the wrong customer's answer
Language model	Hosted model first, local model as backup, then quote the source	One model	Fast and better when a key is set, yet the project runs with no key and no cost

Where the measurements changed the design

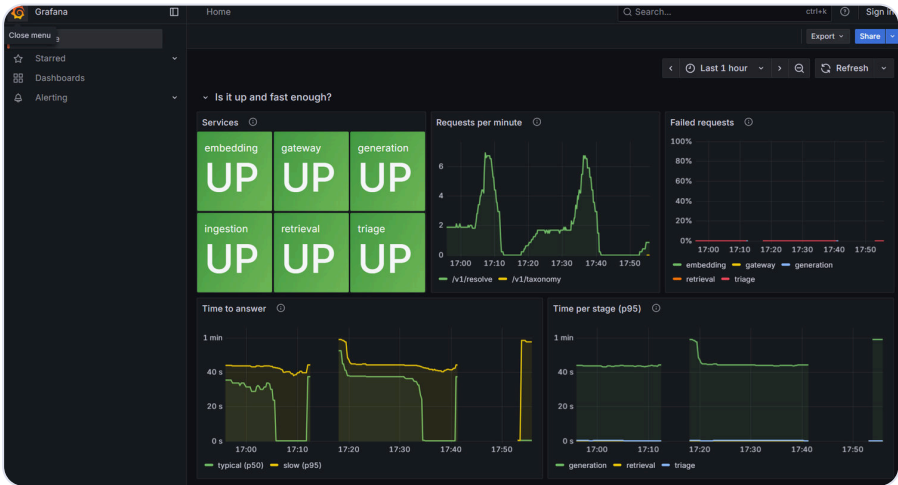
PLAN	WHAT THE MEASUREMENT SAID	WHAT CHANGED
Hybrid search plus a reranker	Equal-weight hybrid was worse than meaning alone; the reranker added 900 ms for no gain	Meaning weighted 3x, reranker off
Incident settings by a first guess	38% of outages noticed, 16% false alarms	Settings picked by the eval: 71% and 2%
A prompt rule: escalate if the sources do not fit	Ignored 13 times out of 13	A similarity cut-off before the model

Built for production

CONCERN	WHAT IS BUILT
A part fails	Each service can fail without taking the rest down. Model down: the backup model answers, then steps are quoted from the source. Triage down: answer without labels. Cache or database down: still answer
Security and privacy	API keys with separate agent and admin rights, a rate limit, sign-in with three roles, and personal details masked before anything is stored or sent to a model
Data that changes	New tickets, edited articles and new classes go live through the API, by a queue with retries and a safety sweep. Old cached answers are dropped automatically
Trust in the answer	Citations are checked against the sources, unsupported steps are flagged, and the customer reply uses only steps that passed
Knowing it is healthy	19 alert rules with their own tests, a 28-panel dashboard, a one-command health check, one log line per request with the same ID in every service
Knowing it is correct	Six eval scripts, 363 fast tests, 24 tests against the running system, CI on every push. In live use: how often agents do what the assistant suggested

At a larger scale

AREA	NOW (ONE LAPTOP)	AT SCALE
Scaling	One container per service	Services hold no state: several copies behind a load balancer
Language model	Small local model, or a hosted one	GPU serving or a hosted model; stream the answer
Search index	One Qdrant node	One million tickets is about 1.5 GB: still one node. Beyond that, sharding and replication
Ingestion	Redis Streams, one worker	More workers in the same group; Kafka at very high volume
Security	Own sign-in, API keys in settings	Single sign-on, a secret manager, encryption at rest



The monitoring dashboard: up and fast enough, answer quality, drift, language model and cache, new data.

Known limits

- **Search is the weak link.** The right source is among those given to the model for about four complaints in five. Almost every wrong answer starts there.
- **It does not always know when it has no fix.** For a telecom problem the knowledge base does not cover, the local model drafts a confident wrong answer; the larger model refuses half of them. A person must review every draft.
- **The customer reply is not measured yet.** Rules check its form and it only receives checked steps, but its wording has no eval.
- **Incident detection only reads the complaint text.** It is a hint for a person, not an outage system.
- **Sign-in is basic.** Demo accounts with one public password, no single sign-on, no password reset.
- **The data is synthetic,** built from 40 hand-written problem scenarios. Real tickets are messier.
- **Small samples** in the answer eval (20 known complaints). The numbers show direction, not precision.

Run it

You need Docker Desktop and Ollama. No paid API and no API key.

```
ollama pull llama3.2:3b
cp .env.example .env                                # Windows: copy .env.example .env
docker compose up -d --build
docker compose ps                                    # every service should show "healthy"
docker compose run --rm tools python scripts/seed.py
```

Then open `http://localhost:8501` and sign in. Three demo accounts share the password `demo1234` :

USERNAME	ROLE	SEES
priya	Agent	Resolve, Feedback, and only their own cases
arun	Second-line expert	The same, plus Record a fix and every agent's cases
meera	Engineer	Everything, plus the Monitoring link

Where to read more

DOCUMENT	WHAT IS IN IT
README.md	What it is, how to run it, what to try, what was measured
docs/ARCHITECTURE.md	The design in plain words, the diagrams, how each requirement is met, scale considerations
docs/DESIGN_DECISIONS.md	Each choice, the alternative, and the measurement behind it
docs/GUIDE.md	Every address and command: API calls, evals, monitoring, tests